

CCF 355: UFV Wilds - Parte 03

Emanuel Vitor Carvalho Ruella (3891)

Marcus Vinicius Guerra Ribeiro (4240)

Alan Gabriel Martins Silva (4663)

¹Instituto de Ciências Exatas e Tecnológicas,
Universidade Federal de Viçosa, Florestal, MG, Brasil

1. Introdução

Este trabalho apresenta a terceira etapa do desenvolvimento do projeto de sistemas distribuídos proposto na disciplina CCF355, com o objetivo de desenvolver um sistema baseado no conceito do jogo Super Trufo, onde os usuários podem colecionar e jogar com cartas que representam personagens, lugares ou objetos.

Essa segunda etapa faz algumas correções, além de implementar o código servidor e o código cliente utilizando-se de Sockets em Python.

2. Proposta de Implementação

A Implementação proposta é o jogo UFV WILDS, baseado no jogo Super Trufo, com 3 jogadores disputando para tomar todas as cartas dos oponentes primeiro.

Para comunicação entre cliente e servidor, utilizamos sockets com configurações específicas como STILLALIVE, e Timeout ativos.

A interface gráfica foi desenvolvida utilizando a biblioteca Pygame.

O servidor foi implementado com as principais bibliotecas:

- `argparse`: Facilita a criação e interpretação de comandos de inicialização do servidor (ex.: `./server -H -dl`).
- `socket`: Responsável pela comunicação em rede entre cliente e servidor.
- `json`: Permite a manipulação e troca de dados em formato JSON.
- `threading`: Garante a execução de múltiplas threads simultaneamente.
- `hashlib`: Utilizada para a criptografia de senhas dos usuários.
- `base64`: Converte imagens PNG para bytes para envio via rede.
- `jwt`: Cria tokens de autenticação para segurança das transações entre cliente e servidor.
- `sqlite3`: Implementa um banco de dados leve para armazenar informações do jogo e do usuário.

Além de outras bibliotecas como `OS`, `TIME`, `DATETIME` entre outros.

O cliente do jogo foi implementado principalmente com a biblioteca `SOCKETS`, `THREADING`, `JSON`, `BASE64` e `Pygame`, possuindo porém outras bibliotecas auxiliares como `TIME`, `HASHLIB`, etc.

3. Servidor

O servidor é dividido em 3 Threads principais: a Thread Main (Utilizada para receber conexões sockets e criar threads individuais para cada conexão), a Thread de terminal, que abre um terminal para o servidor utilizado para certos comandos de administrador, e a Thread de partida, utilizado para conectar os usuarios que requisitarem uma partida para iniciar uma.

A seguir, vamos explicar um pouco sobre a Thread Terminal, e então, seguir o fluxo de funcionamento do servidor.

3.1. Thread Terminal

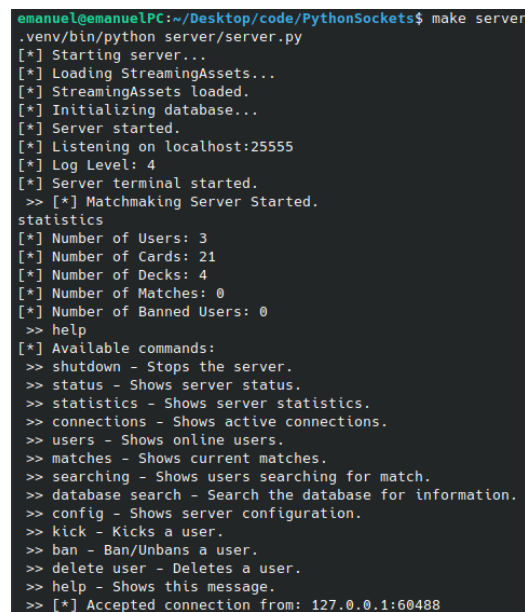
A terminal window showing the execution of a 'make server' command. The output displays various status messages such as 'Starting server...', 'Loading StreamingAssets...', 'StreamingAssets loaded.', 'Initializing database...', 'Server started.', 'Listening on localhost:25555', 'Log Level: 4', and 'Server terminal started.'. It then shows 'Matchmaking Server Started.' followed by a list of statistics: 'Number of Users: 3', 'Number of Cards: 21', 'Number of Decks: 4', 'Number of Matches: 0', and 'Number of Banned Users: 0'. A 'help' command is entered, displaying a list of available commands: 'shutdown', 'status', 'statistics', 'connections', 'users', 'matches', 'searching', 'database search', 'config', 'kick', 'ban', 'delete user', and 'help'. The terminal ends with the message '[*] Accepted connection from: 127.0.0.1:60488'.

Figura 1. download de arquivos pós login

A Thread de Terminal [internal_server_terminal] é uma parte separada do servidor, existente para a configuração do mesmo.

Essa Thread é basicamente uma central de comandos que o Administrador do sistema pode utilizar, com alguns comandos:

- **shutdown:** Desliga o servidor graciosamente, fechando as conexões, threads e saindo do programa.
- **status:** Mostra alguns status do servidor, como o numero de sockets, o numero de usuários logados, o numero de partidas, e o numero de usuários procurando por uma partida.
- **statistics:** Estatísticas internas do servidor, como numero de cartas, numero de usuarios registrados, entre outros.
- **config:** Utilizado para configurar o servidor, como o nivel de Log a ser mostrado na tela, adição de cartas, deletar cartas, e checar cartas no banco de dados.
- **kick e ban:** Utilizado para remover um usuario ou banir o mesmo.
- **delete user** usado para deletar a entrada de um cliente do banco de dados;

Ao todo, a Thread Terminal é apenas para uso interno, e todas as funções que ela utiliza são próprias para ela, ou seja, não são acessíveis por fora do sistema.

3.2. Thread Principal e Funcionamento do programa

Ao executar o servidor, alguns parametros podem ser passados para o mesmo:

- **-H:** Host ou IP, por padrão roda no localhost, mas pode ser definida.
- **-p:** Porta, a padrão e 25555.
- **-l:** Nivel de Log. Por padrão é nivel 4, que vai imprimir diversas informações sobre as conexões e o estado do servidor na tela. O nível máximo é 5, na qual informação de Debug também aparecem no terminal do servidor.
- **-dl:** O nível de Delay, essa configuração é utilizada para o envio de mensagens durante partidas. Por padrão e 0.1, porém, caso o programa rode localmente na mesma maquina, pode ser colocado para 0. O motivo para a existência dessa variável se da ao fato de que durante a execução do servidor em uma maquina externa com 500ms de ping (Servidor hospedado em Frankfurt), o envio imediato de mensagens causava perda de mensagens e até mesmo problemas do buffer do servidor de tunelamento. O problema ocorre graças a arquitetura da rede, porém, um delay de 0.1-0.2ms se mostra suficiente para resolver esse problema nesse caso.

Após o inicio do servidor, as seguintes etapas ocorrem na Thread Principal:

1. O carregamento da pasta StreamingAssets, com nome de arquivos e CheckSum de cada um dos mesmos salvos. (Importante para o envio das cartas e arquivos do jogo aos usuários)
2. A inicialização do Banco de Dados com a Criação do arquivo .DB se não existir, criação das tabelas, checagem de estatísticas, atualização de estatísticas e de quantidade de cartas do servidor.
3. Criação da conexão de Socket utilizando o host e a porta, e um timeout de 5 segundos (Utilizado para verificar, a cada 5 segundos, se o servidor está finalizando com a flag "stop_event")
4. A inicialização da Thread Terminal
5. A inicialização da Thread de Partida;
6. A abertura e verificação de sockets pedindo conexão (socket.accept()), dentro de um While loop que, enquanto o evento de parada não for ativo, ele vai continuar executando.
7. A aceitação da conexão, e criação de uma Thread "Handle Client), para a manipulação de mensagens vindas do cliente.

Esses 7 passos fazem a inicialização do processo servidor, com as proximas etapas ocorrendo nas proximas sessões.

3.3. Thread de Partida

A Thread de partida roda no servidor, ela é considerada como o "servidor de partidas", e executa da seguinte forma:

1. E criado um loop que funciona até o "stop_event" ser chamado
2. Ele fica verificando se ao menos 3 jogadores estão procurando partida na lista de "Procurando por partida"
3. Ao achar 3 jogadores, os mesmos são agrupados e removidos da lista de busca
4. Ele cria uma nova thread, chamada de "Play.Field" e manda os jogadores e seus sockets para la.

3.4. Handle Client

Ao aceitar a conexão com um cliente, algumas configurações de Sockets são feitas como: Colocar um timeout de 5 segundos, e ativar a opção de socket `KEEPALIVE`, com intervalo de 3 segundos entre chamadas, e tempo ocioso de 10 segundos. Esse Keep Alive é utilizado em caso se tenha suspeitas que a conexão do cliente foi perdida.

O cliente então é salvo em uma lista de conexões ativas, e então se espera uma mensagem do cliente. A cada 5 segundos, o timeout se ativa para verificar se o `stop_event` ocorreu, caso contrario, ele volta a esperar mensagem do cliente.

A seguir, o seguinte caminho é seguido:

1. Quando o usuário manda uma mensagem, ele envia essa requisição para a função `handle_response`.
 - `login`: Função de login, faz os procedimentos necessários para login, envia ao cliente status: 200 e um token em caso de sucesso.
 - `register`: Função de Registro, apenas retorna sucesso caso o registro seja sucedido.
 - `funções do jogo`: (`Check_card`, `check_deck`, etc.): Essas funções verificam se receberam um token valido (Ou seja, um usuário autenticado) e retornam as informações necessarias.

Todas as funções acima retornam um JSON com as informações: `"Status"`, `"Message"` e `"Command"`, além de informações adicionais como `"card"` ou `"token"` caso seja requisitado

Quando um usuário loga no sistema, ele é adicionado em uma lista de usuários logados, e quando o mesmo requisita uma partida, ele é adicionado em uma lista de usuários procurando partida.

4. Banco de Dados

O Banco de dados do servidor possui as seguintes tabelas:

- **Users**: Salva informações sobre o usuario, como nome, senha (Hash), se é ou não admin, entre outras informações.
- **card**: Salva informações sobre a carta, além dos valores de seus atributos, o nome do seu arquivo de imagem.
- **user_cards**: A conexão entre usuario - Cartas adquirida, e a quantidade daquela carta (podendo chegar a 3)
- **decks**: Com ID de usuario, se está ativo, ou se é default
- **deck_cards**: Conexão entre cartas e um deck, com 9 cartas permitidas por deck.
- **system_config**: Salva configurações do servidor.
- **statistics**: Salva informações de estatística do servidor.

4.1. Segurança do Banco de Dados

Algumas ações foram tomadas para adicionar um pouco de segurança ao banco de dados, como a atribuição e salvamento de senhas dos usuarios utilizando de HASH SHA256 da biblioteca `HASHLIB`, é de criação e validação de TOKENS utilizando da biblioteca `JWT`, para criar um token que salva, de forma criptografada, o nome do usuario e o tempo de expirar de um token. Esse Token é criado e enviado ao cliente durante o Login, e deve ser enviado para o servidor durante o funcionamento para validar que o usuario está realmente autenticado e valido.

4.2. Funções do Banco de Dados

A maioria das funções da biblioteca fazem chamadas e verificações (try/except, ou verificação de tokens) para o banco de dados (database.db).

5. Cliente

O cliente é dividido em duas partes, o sistema interno para comunicação e sockets, e as telas, feitas em pygame.

5.1. Partes Internas

A parte interna do Cliente, funciona de forma similar ao servidor. A primeira coisa que ocorre ao iniciar o sistema, é verificar se tem um ARGV para selecionar o HOST e a PORTA e a abertura de message_queues para comunicar entre a parte interna e a de telas. Após isso, as seguintes etapas são feitas:

1. O Diretório StreamingAssets é feito caso não exista.
2. O socket do cliente é iniciado, tentando se conectar ao servidor.
3. A conexão é realizada
4. Com a conexão realizada, a thread de cliente é aberta
5. A thread de Recebimento de mensagens é aberta
6. Enquanto o sistema não for finalizado (stop_event), o sistema continuara executando.

A thread de cliente, configura o socket e é utilizado para enviar mensagens para o servidor.

5.1.1. Empacotamento de mensagens

Antes de enviarmos a nossa mensagem, ela é tokenizada e organizada em um JSON, com os padrões de "Token", "Message", "Commands" e qualquer outro atributo necessário acoplado, antes de ser enviado para o Servidor.

5.1.2. Recebimento de Mensagens

O sistema de recebimento de mensagens funciona de forma similar ao de recebimento de mensagens do servidor, com o pacote recebido pelo servidor sendo descompactado, e então, de acordo com o comando emitido pelo servidor, uma ação é realizada (Login, logoff, etc.) na qual alguma informação será salva ou impressa na tela.

5.2. Interface Gráfica

Para a construção da interface gráfica foi escolhida a biblioteca pygames devido a sua praticidade e vasta quantidade de conteúdos online como tutorias, documentações e exemplos práticos.

Para atender as necessidades do trabalho prático e dos casos de uso, foram elaboradas um total de 12 telas diferentes, que seguem o fluxo de acordo com o diagrama 2.

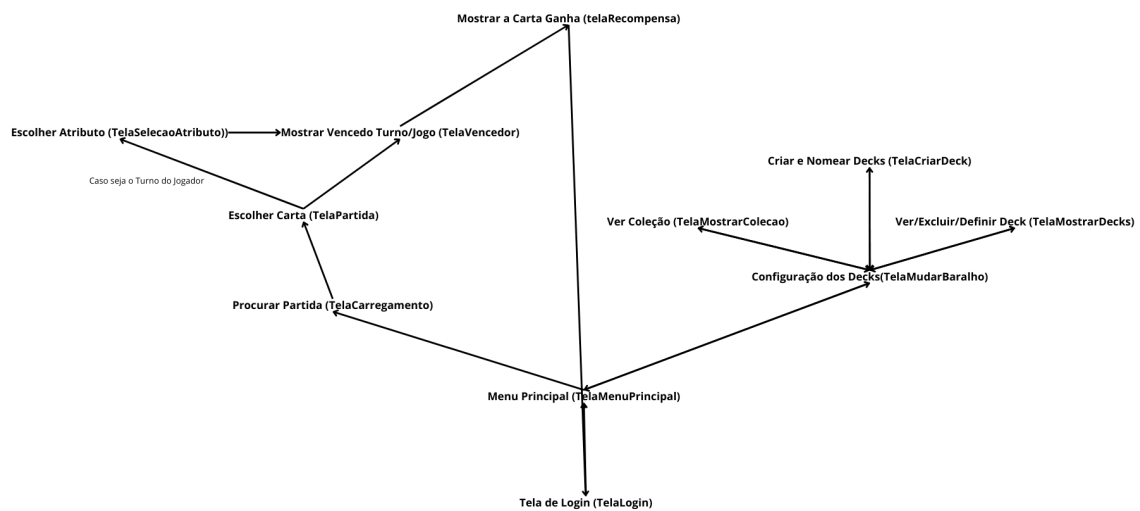


Figura 2. Diagrama de Telas

O processo inicia com o usuário realizando o login ou o registro na TelaLogin. Em seguida, ele acessa o menu principal na TelaMenuPrincipal, onde pode optar por iniciar um jogo. Após selecionar essa opção, o usuário será direcionado para a TelaCarregamento para procurar uma partida.

Quando a partida começa, o usuário escolherá a carta a ser utilizada no turno na TelaPartida. Se for a sua vez de jogar, ele selecionará o atributo desejado na TelaSelecaoAtributo. Após cada turno, a TelaVencedor mostrará o vencedor do turno. O jogo então retorna para a escolha das cartas. Se for o último turno, a TelaVencedor exibirá o vencedor do jogo e, em seguida, irá exibir a carta que o jogador ganhou na telaRecompensa e retornará ao menu principal.

Além disso, o usuário pode optar por editar seus decks na TelaEscolherBaralho. Nessa tela, ele pode ver, definir ou excluir seus decks (TelaMostrarDecks), criar e nomear um novo deck (TelaCriarDeck), e visualizar sua coleção de cartas (TelaMostrarColecao) antes de retornar ao menu principal.

Como a proposta criativa do UFV Wilds é destacar animais e elementos da natureza presentes em Florestal, utilizamos três imagens de fundo 5.2 diferentes nas telas, todas capturadas em locais distintos de Florestal pelos membros do grupo.

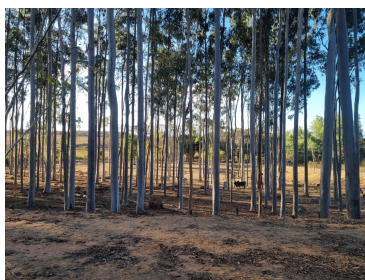


Figura 3. Campo



Figura 4. Cerrado

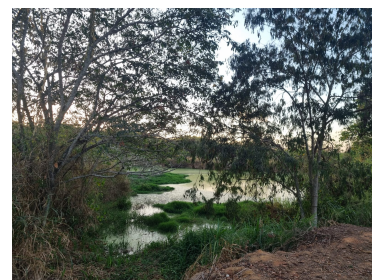


Figura 5. Pântano

Figura 6. Imagens de Fundo

6. Funcionalidades

6.1. Registrar e cartas iniciais

Quando um usuário se registra no servidor, o banco de dados faz algumas operações:

- Ele cria o usuário
- Ele coloca suas estatísticas como 0
- Ele aloca 9 cartas aleatórias para o usuário (Cartas únicas)
- Ele cria um Deck Default.
- Ele coloca o Deck Default como o deck ativo.

Após isso, quando o usuário entrar no jogo, ele já estará pronto para jogar. É possível ter cartas repetidas em um mesmo deck (até 3 cartas), porém, para ser possível todas as cartas entrarem em circulação, as primeiras cartas de todos os jogadores são sempre aleatórias e únicas, sem cópias.

6.2. Login e Download de arquivos

```
[*] Loading... Please wait
[*] There exists 22 files.
[*] Missing file: passaro_preto_e_branco.png
[*] Missing file: garca_moura.png
[*] Missing file: sapao_faraonico.png
[*] Missing file: grilo.png
[*] Missing file: passaro_pequeno.png
[*] Missing file: quati.png
[*] Missing file: coruja.png
[*] Missing file: calopsita.png
[*] Missing file: sapinho_pequeno.png
[*] Missing file: quero_quero.png
[*] Missing file: background.png
[*] Missing file: aranha.png
[*] Missing file: mariposa_vermelha.png
[*] Missing file: cobra_de_vidro.png
[*] Missing file: garca_branca.png
[*] Missing file: bem_te_vi.png
[*] Missing file: lagartixa.png
[*] Missing file: siriemas.png
[*] Missing file: mariposa_enorme.png
[*] Missing file: louva_a_deus.png
[*] Missing file: tucano.png
[*] Missing file: borboleta.png
[*] Missing 22 files.
[*] Requesting file: passaro_preto_e_branco.png 0 / 22
[*] File /home/emanuel/Desktop/code/aaa/StreamingAssets/passaro_preto_e_branco.png downloaded.
[*] Requesting file: garca_moura.png 1 / 22
[*] File /home/emanuel/Desktop/code/aaa/StreamingAssets/garca_moura.png downloaded.
[*] Requesting file: sapao_faraonico.png 2 / 22
[*] File /home/emanuel/Desktop/code/aaa/StreamingAssets/sapao_faraonico.png downloaded.
[*] Requesting file: grilo.png 3 / 22
[*] File /home/emanuel/Desktop/code/aaa/StreamingAssets/grilo.png downloaded.
[*] Requesting file: passaro_pequeno.png 4 / 22
[*] File /home/emanuel/Desktop/code/aaa/StreamingAssets/passaro_pequeno.png downloaded.
[*] Requesting file: quati.png 5 / 22
[*] File /home/emanuel/Desktop/code/aaa/StreamingAssets/quati.png downloaded.
[*] Requesting file: coruja.png 6 / 22
[*] File /home/emanuel/Desktop/code/aaa/StreamingAssets/coruja.png downloaded.
[*] Requesting file: calopsita.png 7 / 22
```

Figura 7. download de arquivos pós login

Ao logar no servidor, o usuário recebe uma lista de arquivos e seus Checksums para verificar a integridade dos arquivos locais da pasta StreamingAssets. O cliente então, checa essa lista, verifica com a que possui (dos próprios arquivos locais), e envia para o servidor quais arquivos estão faltando ou corrompidos. O servidor então envia os arquivos para o cliente, pedaço por pedaço. Ou seja, todas as imagens necessárias para o funcionamento do jogo.

Vale apontar que as imagens são enviadas dentro de um arquivo JSON em pedaços de 4096 bits, e esses pedaços são agrupados e quando o JSON é finalizado, ele é então lido e executado, convertendo a imagem (Agora em b64) para PNG.

6.3. Comandos de Deck.

Após o Login, algumas funções ficam disponíveis para o cliente, como, checagem de deck (check deck), checagem de cartas (check card), entre outras funções que necessitam do usuário estar autorizado.

Todo usuário possui um deck inicial, chamado de "Default", esse deck, não pode ser deletado ou modificado, e por padrão está sempre ativo.

Em caso do usuário deletar um deck ativo, o deck Default se torna ativo por padrão.

7. Campo de jogo

O sistema de partidas começa após 3 usuários autenticados colocarem a busca por partidas. A Thread de partidas cria o PlayField. Dentro do PlayField, as seguintes operações ocorrem:

1. As informações dos jogadores são obtidas (ID, Deck ativo, Cartas no Deck)
2. As cartas na mão de todos os jogadores é aleatorizada em ordem
3. A ordem de jogada e selecionada aleatoriamente e salva.
4. Uma mensagem é enviada que a partida foi iniciada para os jogadores.
5. Três cartas são dadas para cada jogador.
6. Um loop ocorre, na qual a partida em si ocorre.
7. Um turno acontece, na qual o primeiro jogador da ordem é selecionado para selecionar um atributo que será jogado, e depois a carta
8. Os outros dois jogadores jogam uma carta.
9. Após isso, o jogador é determinado com a seguinte ordem: O jogador maior é selecionado aleatoriamente em caso de empate (entre as cartas que empataram), ou o jogador com o melhor valor de atributo ganha por padrão.
10. O jogador que ganha, adiciona todas as 3 cartas em seu monte.
11. A ordem de jogo avança, com o primeiro jogador indo para a posição de jogada do terceiro jogador, terceiro jogador indo para a posição do segundo jogador, e o segundo jogador indo para a posição do primeiro jogador.
12. Após isso, os jogadores compram uma carta para manterem a mão com 3 cartas.
13. Caso algum jogador tenha 0 cartas restantes no monte, a partida acaba, com o jogador que tem mais cartas ganha.

Após isso, a sala de partida é finalizada, a thread fecha, e os jogadores voltam para o lobby.

8. Troca de informações entre a thread de interface gráfica e thread de comunicação com o servidor

A comunicação com o cliente e servidor e a interface gráfica são executadas em threads diferentes. Para estabelecer uma comunicação entre essas threads e as ações do servidor serem mostradas na interface gráfica e para que as ações realizadas na interface gráfica fossem enviadas para o servidor, foram usadas Queues. Uma Queue é uma estrutura da linguagem Python que funciona como uma fila(queue) global, ou seja, seu conteúdo é compartilhado entre as threads. Então, foram usadas duas queues: uma para enviar informações da thread interface gráfica para a thread comunicação com o servidor, e uma para enviar as mensagens que eram recebidas do servidor pela thread de comunicação para a thread interface gráfica.

9. Observações sobre a Implementação

Ao final dessa entrega, a implementação via terminal do jogo UFV_WILDS se torna concluída e a implementação via interface gráfica também! Foi um desafio interessante essa entrega, mas, com um pouco de esforço, deu tudo certo.

Existem três contas pre-registradas no banco de dados da entrega: mixxs, marcus e alan, todas com a senha 123456.